

Documentation1

The two cameras see the object from two different positions; the difference is called **disparity**. From disparity we can calculate how far away the object is. (depth)

Once we know the 3D coordinate, we can calculate the 3D distance between them.

That's object length.

Part One: Calibration:

Stores the information given here:

https://docs.google.com/document/d/19_eeMn2VCZcnhHVvlBt8LIY2vI1zB-p-/edit

<https://docs.google.com/document/d/1B2EBcO-7fC7PPzHf-mKYVzooqMUOH2n0/edit>

<https://docs.google.com/document/d/1xPDNALYqhwy5qzDJnbvILRsFUdn0Qih/edit>

ex: pair 0:

```
0: {  
  "K_left": np.array(  
    [5299.313, 0, 1263.818],  
    [0, 5299.313, 977.763],  
    [0, 0, 1]  
  ), dtype=np.float64),  
  "K_right": np.array(  
    [5299.313, 0, 1438.004],  
    [0, 5299.313, 977.763],  
    [0, 0, 1]  
  ), dtype=np.float64),  
  "doffs": 174.186,  
  "baseline_mm": 177.288,  
  "width": 2988,  
  "height": 2008,  
  "ndisp": 180,  
  "vmin": 54,  
  "vmax": 147  
},
```

K: camera intrinsic matrix.

Describing the camera itself.

$$\begin{bmatrix} f_x & 0 & c_x \end{bmatrix}$$

$$\begin{bmatrix} 0 & f_y & c_y \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 1 \end{bmatrix}$$

Focal length and center of the camera.

Baseline: Physical distance between the 2 cameras.

Doffs = disparity offset.

$$Z = \frac{B \times F}{Doff + D}$$

Disparity = what the cameras measured.

Disparity offset (doffs) = a calibration correction added to that measurement.

A nearby object produces a larger difference between its positions in the two camera images, while a distant object produces a smaller difference. Therefore, large disparity means small depth, and small disparity means large depth.

SIFT:

`Sift = cv2.SIFT_create()`

It looks for the interesting features in the images.

Then we match them.

`Matcher = cv2.BFMatcher()`

SIFT gives each possible match a score

Ex:

Best match has a distance of 20 and second has a distance of 50.

According to Lowe ratio test:

```
if m.distance < 0.75 * n.distance:
```

```
    good_matches.append(m)
```

$$\frac{20}{50} = 0.4$$

$$20 < 0.75 * 50$$

$$20 < 37.5$$

So, we kept that match.

But what if best and second-best are close and the best isn't much better than the second.

We rejected it.

Because the best match isn't much better than the second-best match. The algorithm isn't confident that it's the same feature.

But not all sift matches can be correct, so we use **RANSAC**.

It chooses a random subset, calculates a model and sees how many other matches agree with that model. Because corresponding points from the camera obey a specific epipolar geometry.

```
matcher = cv2.BFMatcher()

matches = matcher.knnMatch(
    des_left,
    des_right,
    k=2
)

good_matches = []

for pair in matches:
    if len(pair) != 2:
        continue

    m, n = pair

    if m.distance < 0.75 * n.distance:
        good_matches.append(m)

print(
    "Good matches:",
    len(good_matches)
)
```

SIFT → best & second-best → reject uncertain matches → reliable matching points
→ Fundamental Matrix.

SIFT is used to find distinctive features in the left and right images and determine which features correspond to each other. For every feature in the left image, SIFT searches for the most similar features in the right image. Instead of considering only the best match, we consider the best and second-best matches to check how reliable the match is. The distance between two features represents how different they are, so a smaller distance means the features are more similar. If the best match has a much smaller distance than the second-best match, we can be confident that it is the correct match. However, if the two distances are very close, the algorithm cannot confidently determine which feature is correct, so the match is rejected. This is done using Lowe's Ratio Test, where the best match distance is compared with 75% of the second-best match distance. This filtering step removes unreliable matches and provides more accurate feature points for calculating the Fundamental Matrix and the rest of the stereo vision pipeline.

Gives us corresponding points $(x,y)-(x,y)$

Fundamental Matrix:

Now we have corresponding points filtered by RANSAC.

We feed them into:

`Cv2.findfundemntalmatrix(...)`

It basically tells us where a point's corresponding point can lie in the other image.

It gives us camera geometry between each other.

```
def calculate_fundamental(
    pts_left,
    pts_right
):
    if len(pts_left) < 8:
        raise ValueError(
            "At least 8 matches are required."
        )

    F, mask = cv2.findFundamentalMat(
        pts_left,
        pts_right,
        cv2.FM_RANSAC,
        1.0,
        0.99
    )

    if F is None:
        raise ValueError(
            "Could not calculate Fundamental Matrix."
        )

    mask = mask.ravel().astype(bool)
    inlier_left = pts_left[mask]
    inlier_right = pts_right[mask]

    return (
        F,
        inlier_left,
        inlier_right
    )
```

Epipolar lines:

Instead of searching everywhere we can search along a certain line.

`Cv2.computercorrespondepilines(...)`

The green lines are epipolar lines while the red dots are the feature points.

Essential Matrix:

converts the pixel coordinates of the matched points in the left image to normalized camera coordinates using the intrinsic camera matrix K_{left} . This step is necessary for accurate pose estimation, as it removes the effects of lens distortion which is none in this code and scales the points to a common coordinate system.

Knowing that fundamental matrix + camera calibration = essential matrix.

$$E = K_{RIGHT} T F K_{LEFT}$$

Then

`Cv2.recoverpose(...)`

Gives us rotation and translation.

R = Rotation

How one camera is rotated relative to the other.

T = Translation

The direction in which one camera is located relatively to the other.

```
def calculate_pose(
    F,
    K_left,
    K_right,
    pts_left,
    pts_right
):
    # Essential matrix
    E = (
        K_right.T
        @ F
        @ K_left
    )

    # Normalize points
    left_norm = cv2.undistortPoints(
        pts_left.reshape(-1, 1, 2),
        K_left,
        None
    )

    right_norm = cv2.undistortPoints(
        pts_right.reshape(-1, 1, 2),
        K_right,
        None
    )

    # Recover rotation and translation
    _, R, t, _ = cv2.recoverPose(
        E,
        left_norm,
        right_norm
    )
```

Rectification:

Originally the cameras saw the scene at slightly different angles.

We want to transform the images so corresponding points are on the same horizontal line.

USING:

`Cv2.stereorectify(..)`

`Cv2.initundistortrectifymap(...)`

`Cv2.remap(...)`

Giving us `rect_left` and `rect_right`.

Homography matrices:

A homography matrix is basically a mathematical transformation that tells the computer how to move/transform the pixels of an image.

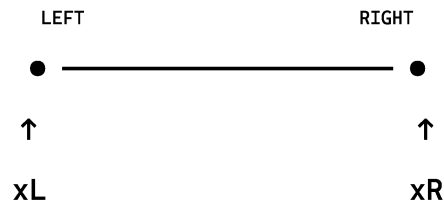
The code calculates `H1` and `H2`.

These describe the transformation that maps the original image into its rectified one.

Has no importance but it gives insight on depth and disparity.

Disparity:

After rectification:



Disparity is $x_L - x_R$.

The closer the object, the larger the disparity.

SGBM:

SGBM = Semi-Global Block Matching.

It is the algorithm we use to find which pixel in the right image corresponds to each pixel in the left image.

SGBM looks at a small window/block of pixels around a point in the left image and searches along the corresponding horizontal line in the right image.

Does this pixel match the right image? Matching cost.

Does this disparity make sense compared to nearby pixels?

For example:

A	B	C	D
10	10	4	10

For c, $\text{disp} = 4$ is smallest, so $\text{SSD } (L-R)^2$ will choose it.

But it's suspicious for sift.

Cross correlations see if they r similar in pattern.

SGBM → finds correspondences → calculates disparity → disparity is used to calculate depth.

USING:

`Cv2.stereoSBGM_create(...)`

It produces disparity.

It basically looks at neighboring pixels/windows and tries to determine:

“Which location in the right image corresponds to this location in the left image?”

```
def get_sift_matches(left, right):
```

```
    gray_left = cv2.cvtColor(  
        left,  
        cv2.COLOR_BGR2GRAY  
    )
```

```
    gray_right = cv2.cvtColor(  
        right,  
        cv2.COLOR_BGR2GRAY  
    )
```

```
    sift = cv2.SIFT_create(  
        nfeatures=3000  
    )
```

```
    kp_left, des_left = (  
        sift.detectAndCompute(  
            gray_left,  
            None  
        )  
    )
```

```
    kp_right, des_right = (  
        sift.detectAndCompute(  
            gray_right,  
            None  
        )  
    )
```

```
    print(  
        "Left SIFT features:",  
        len(kp_left)  
    )
```

Disparity Image:

Raw disparity might look like numbers, so we convert these values from 0-225.

Using

`Normalize_image()`

We normalize the coordinates to improve numerical conditioning and stability during geometric estimation. This makes the Fundamental Matrix

estimation more reliable and reduces the effect of differences in coordinate scale.

Then we create the heatmap:

```
cv2.applyColorMap(  
    disparity_gray,  
    cv2.COLORMAP_JET)
```

Depth:

THIS is where the stereo camera becomes a 3D sensor.

We use:

$$Z = \frac{B \times F}{D_{off} + D}$$

- Z = depth
- f = focal length
- B = camera baseline
- d = disparity
- d_{offs} = disparity offset

For pair 0:

- $f = 5299.313$
- $B = 177.288$ mm
- $d_{offs} = 174.186$
- ASSUMING disparity is 100.

Then the depth is 3.43.

DEPTH:

Now every pixel has a depth.

So, we save these images.

Measuring:

Finally, the user can click two points to measure the distance between them.

$$x = \frac{(u - c_x)Z}{F_x}$$

$$y = \frac{(u - c_y)Z}{F_y}$$

And Z previously.

NOW we have A=(X,Y,Z).

So, the program calculates distance:

$$L = \sqrt{[(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2]}$$

